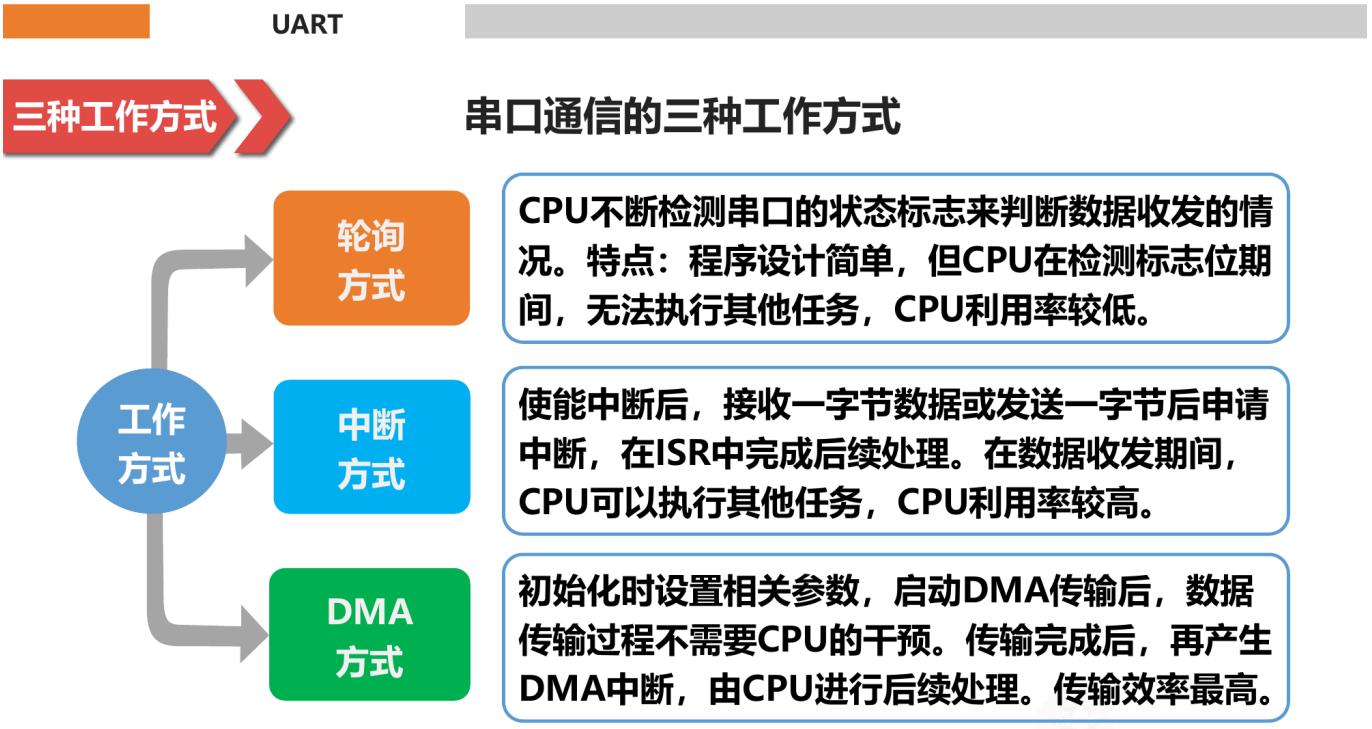
STM32 串口通信实验报告

本实验学习异步串行通信的帧格式、波特率，以及 TTL、RS-232 和 RS-485 接口的信号差异；使用 STM32CubeMX、Keil 和 HAL 库配置 STM32F103C8Tx 的 USART1，并通过 USB-TTL 转换模块与 Windows 串口助手通信。程序先以轮询方式连续发送 `hello windows!`，比较加入延时和不加延时时的接收显示；再以串口接收中断识别上位机发送的 `#` 和 `*`，分别暂停和恢复发送；最后配置 USART1_TX 的 DMA，连续向上位机发送数据。

关键词： UART；USART1；HAL；中断；DMA；波特率；TTL；RS-232；RS-485

1. 实验目的

1. 理解 UART 异步串行通信的基本格式、波特率和收发过程。
2. 区分 TTL、RS-232 和 RS-485 的信号方式与电气电平，理解 USB-UART/电平转换模块的作用。
3. 掌握 STM32CubeMX 中 USART1、NVIC 和 DMA 的基本配置方法。
4. 分别用 HAL 轮询、接收中断和 DMA 完成串口发送/控制实验。
5. 估算串口文件传输时间，并比较理论时间与实测时间。
6. 尝试使用 Keil Logic Analyzer 观察串口发送引脚，并依据波特率计算位时间。



2. 实验原理

2.1 UART 异步串行通信

UART 通信不单独传送时钟线，收发双方事先约定波特率和字符格式。每帧通常包含起始位、数据位、可选校验位和停止位。STM32 与串口助手采用 **115200 bps、8 位数据、无校验、1 位停止位（8N1）** 时，每个字符占用 10 个线路位：1 个起始位、8 个数据位、1 个停止位。UART 数据位通常低位先发送。

波特率：代表**每秒线路上传输的码元位数**。 理想条件下，单个比特的时间宽度：

$T_{bit} = \frac{1}{B}$

式中，B 为波特率。

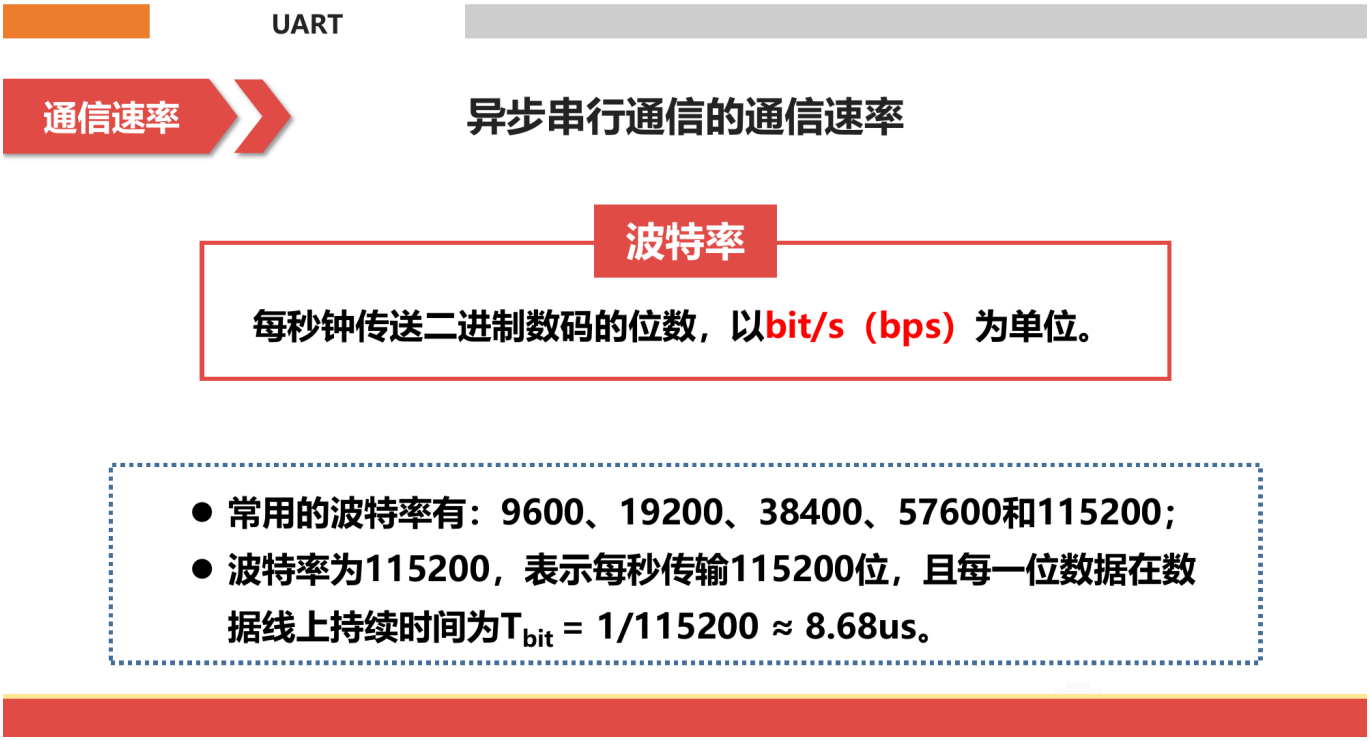
举例：波特率 115200 bps

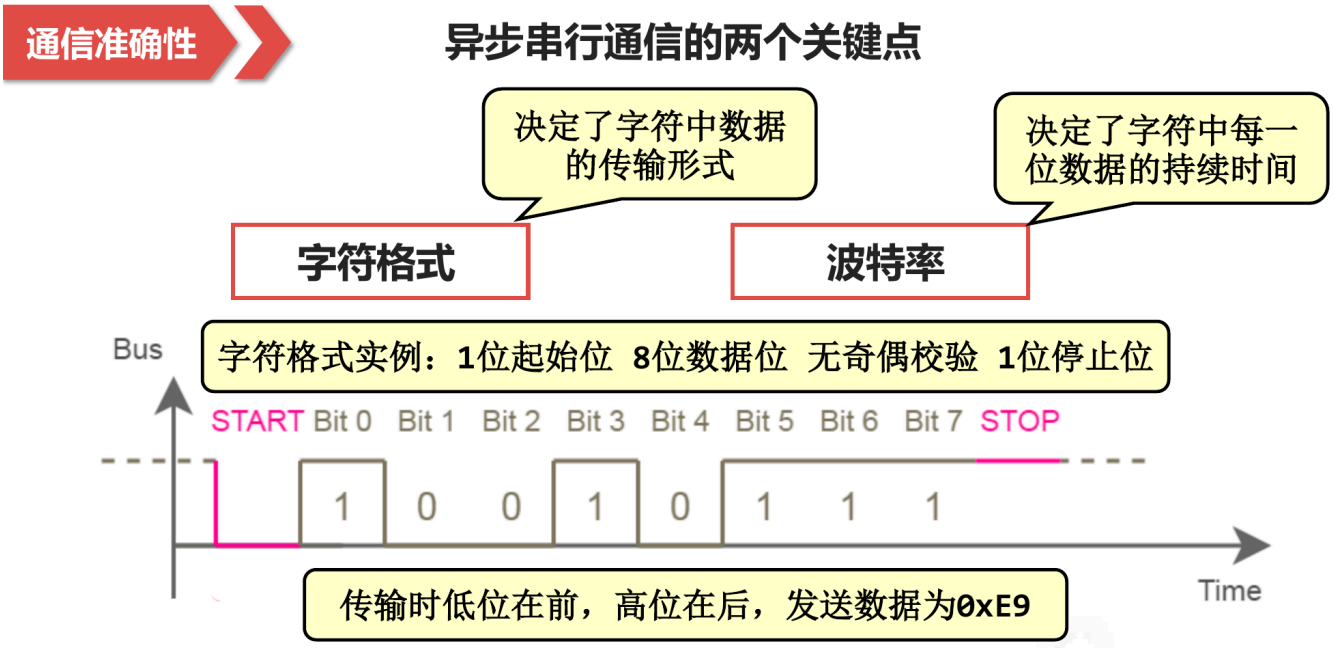
$$T_{bit} = \frac{1}{115200} \approx 8.68\ \mu s$$

串口常用配置 8N1：

- 8：数据位
- N：无校验位
- 1：停止位

传输 1 字节有效数据，额外需要 1 **位起始位** + 1 **位停止位**，一共要传送 10 个比特。 因此**理论有效载荷速率**： $R_{payload} = \frac{B}{10}$ (字节/秒)





2.2 TTL、RS-232 与 RS-485

接口	信号形式	常见电气特征	连接注意事项
TTL/CMOS UART	单端信号，以 GND 为参考	STM32 常用 3.3 V；空闲态为高电平	TX 接对端 RX，RX 接对端 TX，并共地；先确认模块逻辑电压
RS-232	单端、相对信号地的双极性电压，逻辑极性与 TTL UART 相反	接收端通常将低于 −3 V 识别为逻辑 1，将高于 +3 V 识别为逻辑 0；中间区域不作有效逻辑判定	需要 RS-232 收发器进行电平转换；不能把 RS-232 电压直接接 STM32 GPIO
RS-485	A/B 差分信号	接收器按两线间的差分电压判决，抗共模干扰能力较强	需要 RS-485 收发器；多节点总线通常需要匹配终端和合理偏置，不能把 A/B 当作 TTL TX/RX

TTL、RS-232 和 RS-485 不是同一种电平接口。USB-TTL 模块的 TXD/RXD 通常是 TTL/CMOS UART 电平。CH340 是 USB 与 UART 之间的桥接芯片；如果一个产品标称 USB 转 RS-232，它还应包含 MAX232/MAX3232 一类的电平转换器。

2.3 CH340模块

CH340的基本作用是：

```
电脑USB数据
    ↓
CH340
    ↓
TTL UART：TX、RX、GND
```

CH340本身通常不是RS-232电平转换芯片。如果模块要输出真正的RS-232电平，还要带有MAX232、MAX3232或同类收发器：

```
USB → CH340 → TTL UART → MAX3232 → RS-232
```

买模块或接线前一定看清引脚标注：

- 只有 `TXD/RXD/GND/VCC`：通常是USB-TTL。
- 有DB9接口，且板上还有MAX3232：通常是真正USB-RS232。
- 不要只凭商品标题判断。

2.4 串口文件传输时间估算

对于 8N1，忽略协议开销时，大小为 (S) 字节的文件理论传输时间为：

$T_{ideal} = \frac{10S}{B}$ 例如 1 MiB = 1,048,576 字节，在 115200 bps 下：

$T_{ideal} = \frac{10 \times 1{,}048{,}576}{115{,}200} \approx 91.02\ s$

这只是线路传送时间下限。文件传输软件可能加入分包、间隔、校验、应答和重传，因此实测时间通常更长。换算实测有效速率时，可用 S/T_{actual}表示实际有效传输速率，单位为字节/秒；与理论值比较时应统一使用 MB（十进制）或 MiB（二进制）单位。

2.5 轮询、中断与 DMA

- **轮询方式：** CPU 调用 `HAL_UART_Transmit()`，HAL 在函数内部等待发送标志并完成数据发送。编程直观，但调用期间 CPU 一直等待。
- **中断方式：** 调用 `HAL_UART_Receive_IT()` 启动中断接收；收到指定数据后进入 HAL 回调函数处理。接收完一个字节后，需要重新启动下一字节接收。
- **DMA 方式：** CPU 调用 `HAL_UART_Transmit_DMA()` 启动数据搬运，DMA 控制器将内存数据写入 USART 发送寄存器；完成后触发中断，由完成回调通知程序。DMA 可减少 CPU 逐字节搬运工作，但线路波特率仍由 USART 配置决定。

3. 实验设备与连接

3.1 设备与软件

设备/软件	用途
STM32F103C8Tx 开发板	USART1 串口实验对象
USB-TTL 串口模块	STM32 与电脑 USB/串口助手之间的电平与接口转换
杜邦线	TX、RX、GND 信号连接
STM32CubeMX	外设、时钟、中断和 DMA 配置及代码生成
Keil MDK	HAL 程序编译、下载和软件仿真
Windows 串口助手	接收 `hello windows!` 并发送控制字符

3.2 STM32 与 USB-TTL 接线

STM32F103C8Tx	USB-TTL 模块
PA9 / USART1_TX	RXD
PA10 / USART1_RX	TXD
GND	GND

模块通过 USB 连接电脑时，通常由 USB 供电，不需要再把模块 VCC 接到开发板。**信号地 GND 仍要共地**，它为单端 TTL 信号提供共同参考。TX/RX 若没有交叉连接，通常不能正常收发。

3.3 双电脑文件传输接线

两台电脑都安装CH340驱动和SSCOM，设置相同参数：

- 波特率：115200
- 数据位：8
- 停止位：1
- 校验：None
- 流控：None
- 关闭十六进制发送，按二进制原样发送文件
- 关闭自动添加回车换行
- 接收端选择“保存接收数据到文件”

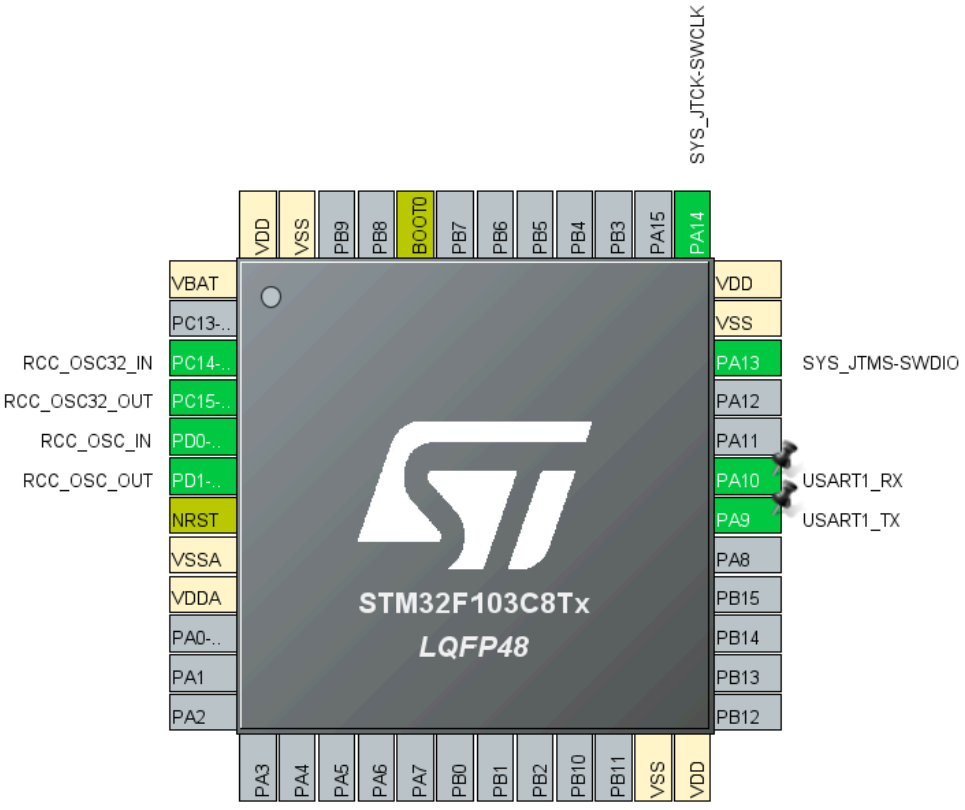
4. 实验步骤与结果

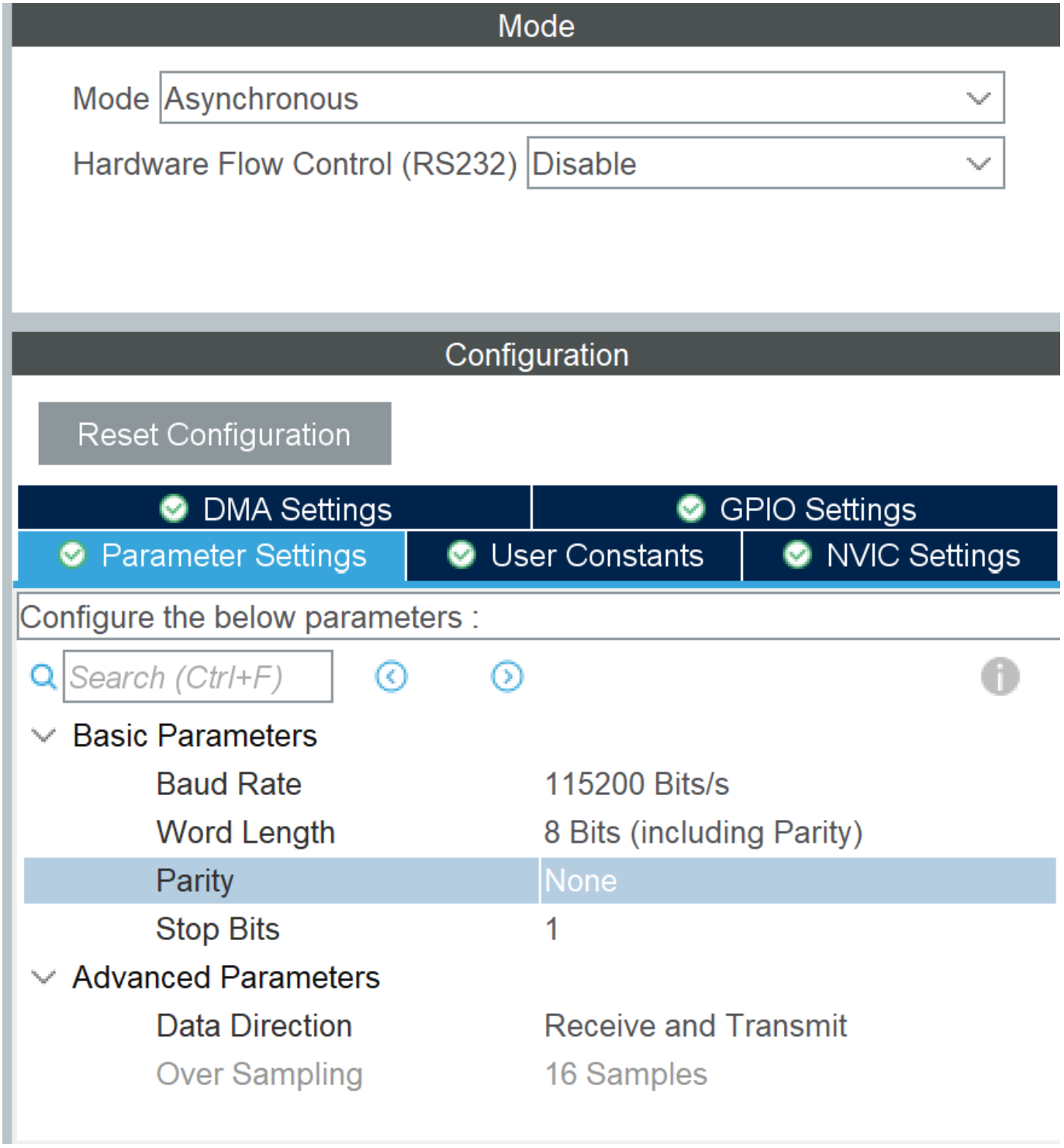
4.1 CubeMX 配置 USART1

1. 在 CubeMX 芯片图上点击右侧的 **PA9**，菜单里选 `USART1_TX`。
2. 点击右侧的 **PA10**，选 `USART1_RX`。
3. 左侧打开 `Connectivity → USART1`，Mode 选 `Asynchronous`。
4. 参数设置为：

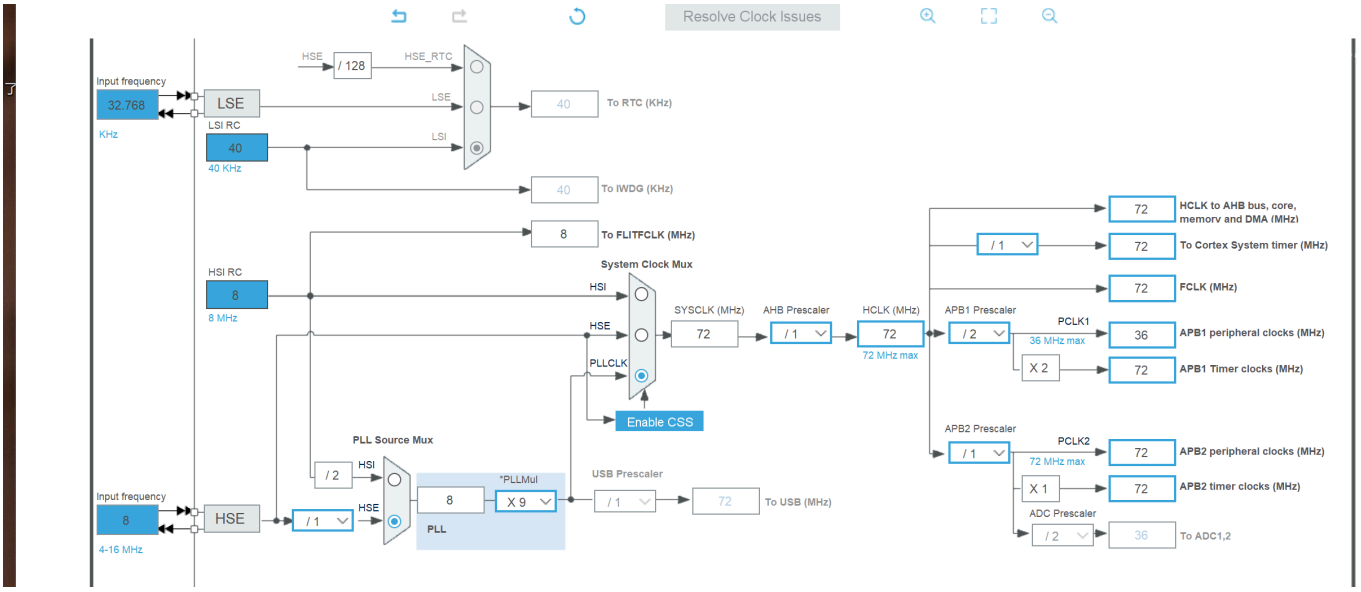
```
Baud Rate: 115200
Word Length: 8 Bits
Parity: None
Stop Bits: 1
Hardware Flow Control: None
```

5. 回到芯片图确认 PA9 显示 `USART1_TX`、PA10 显示 `USART1_RX`，没有变红或冲突。





时钟：当前是 8 MHz 外部时钟经 PLL ×9，系统时钟 72 MHz，APB2 为 72 MHz，符合 STM32F103 常见配置。



4.2 轮询方式连续发送

在 `main.c` 的用户代码变量区加入：

```
/* USER CODE BEGIN PV */
static const uint8_t message[] = "hello windows!\r\n";
/* USER CODE END PV */
```

在 `while (1)` 中加入：

```
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_UART_Transmit(
        &huart1,
        (uint8_t *)message,
        sizeof(message) - 1,
        HAL_MAX_DELAY
    );

    /* 第一次测试加上 */
    HAL_Delay(100);

    /* USER CODE END WHILE */
}
```

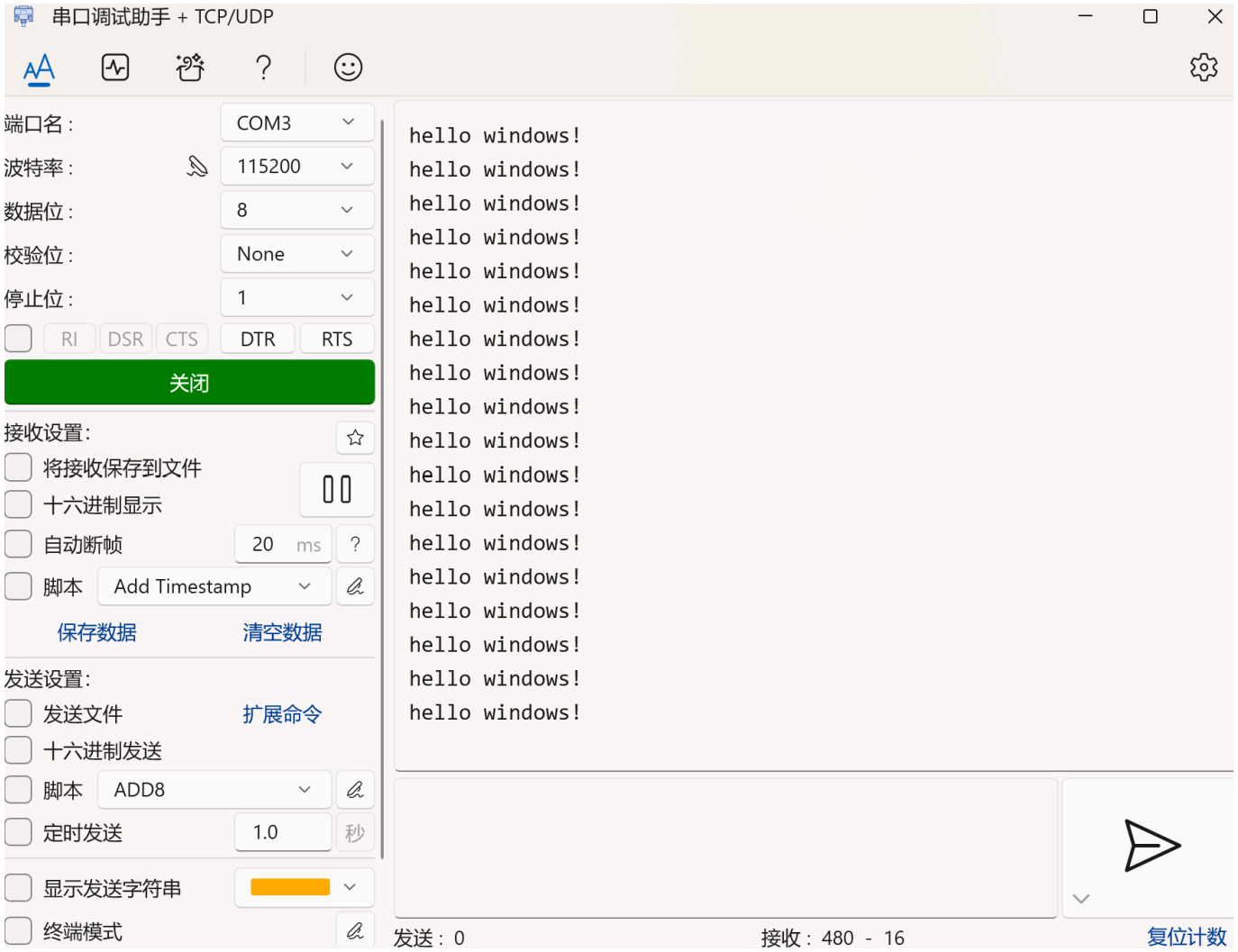
注意：`HAL_UART_Transmit()` 是阻塞发送，它不会瞬间把整段字符串压入线路，而是按115200 bps等待串口逐字节发送。因此删除延时后理论上不应该因为STM32“发送过快”而丢失；但上位机软件刷新、USB缓存或日志保存可能丢数据。

接口函数：HAL_UART_Transmit	
函数原型	HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
功能描述	在轮询方式下发送一定数量的数据
入口参数1	huart：串口句柄的地址
入口参数2	pData：待发送数据的首地址
入口参数3	Size：待发送数据的个数
入口参数4	Timeout：超时等待时间，以ms为单位，HAL_MAX_DELAY表示无限等待
返回值	HAL状态值：HAL_OK表示发送成功；HAL_ERROR表示参数错误；HAL_BUSY表示串口被占用；HAL_TIMEOUT表示发送超时
注意事项	1. 该函数连续发送数据，发送过程中通过判断TXE标志来发送下一个数据，通过判断TC标志来结束数据的发送 2. 如果在等待时间内没有完成发送，则不再发送，返回超时标志 3. 该函数由用户调用

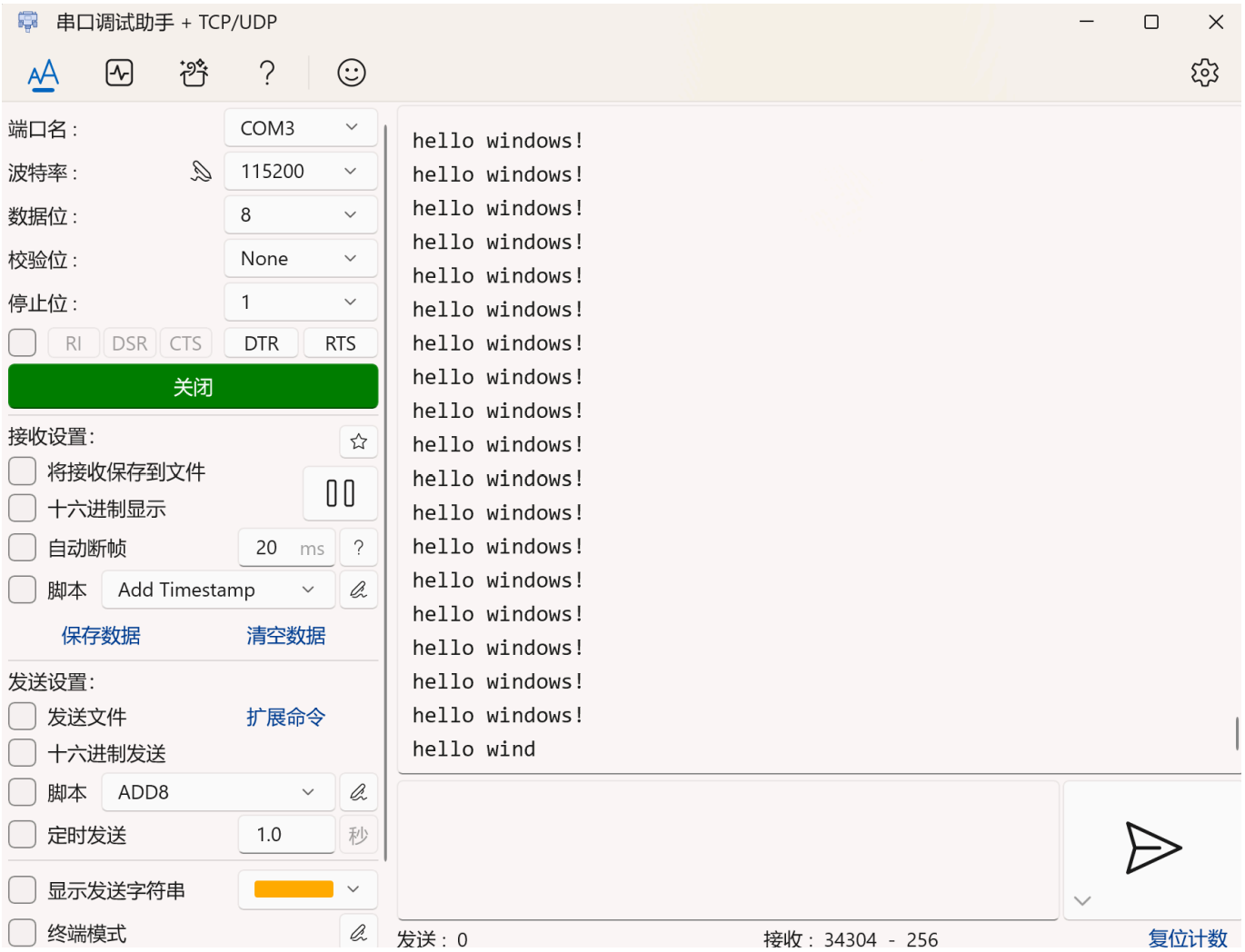
测试	观察结果
有延时	串口助手持续显示完整的 `hello windows!` 行，发送间隔较明显
无延时	串口助手仍能连续看到 `hello windows!`，接收计数持续增长

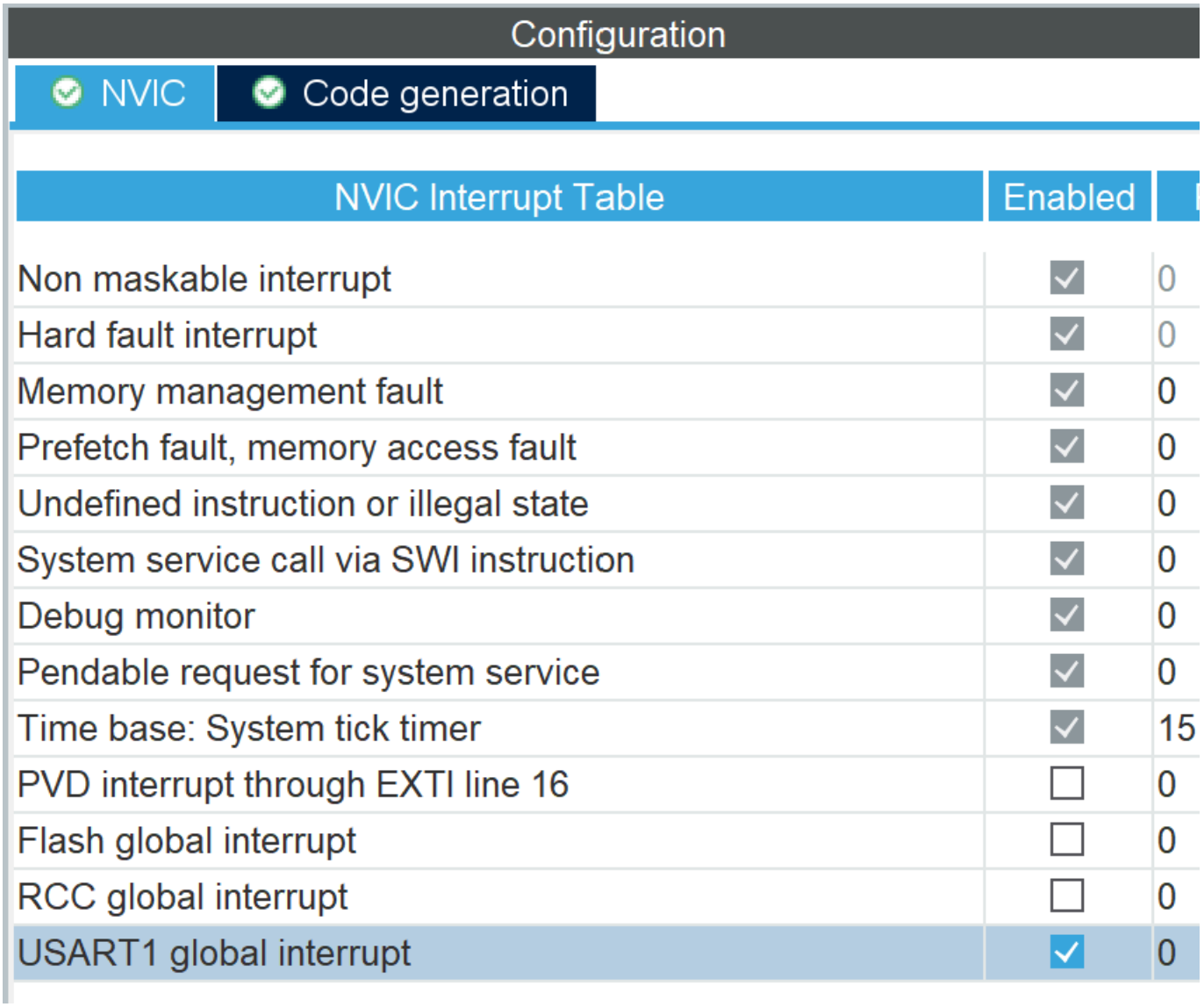
要严谨判断丢失与否，应固定运行时长，将接收内容保存到文件，检查总字节数和重复字符串边界，必要时加入序号或校验值。串口助手刷新/滚屏能力也可能影响窗口观感。

有时延


串口调试助手 + TCP/UDP
端口名：
COM3
波特率：
115200
数据位：
8
校验位：
None
停止位：
1
RI
DSR
CTS
DTR
RTS
关闭
接收设置：
将接收保存到文件
十六进制显示
自动断帧
20
ms
脚本
Add Timestamp
保存数据
清空数据
发送设置：
发送文件
扩展命令
十六进制发送
脚本
ADD8
定时发送
1.0
秒
显示发送字符串
终端模式
hello windows!
发送：0
接收：480 - 16
复位计数

无时延


串口调试助手 + TCP/UDP
端口名：
COM3
波特率：
115200
数据位：
8
校验位：
None
停止位：
1
RI
DSR
CTS
DTR
RTS
关闭
接收设置：
将接收保存到文件
十六进制显示
自动断帧
20
ms
脚本
Add Timestamp
保存数据
清空数据
发送设置：
发送文件
扩展命令
十六进制发送
脚本
ADD8
定时发送
1.0
秒
显示发送字符串
终端模式
hello windows!
hello windows!
hello windows!



先在 `/* USER CODE BEGIN PV */` 和 `/* USER CODE END PV */` 之间添加接收变量和发送开关：

```
uint8_t rx_byte;
uint8_t message[] = "hello windows!\r\n";
volatile uint8_t send_enable = 1;
```

然后把 `/* USER CODE BEGIN 2 */` 里的 `uint8_t message[] = ...` 那一行删掉，改为启动单字节中断接收：

```
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
```

主循环里暂时先只做“允许时发送”。把原来的发送行替换为：

```
if (send_enable)
{
    HAL_UART_Transmit(&huart1, message, sizeof(message) - 1, HAL_MAX_DELAY);
}
```

到 `main.c` 底部的：

```
/* USER CODE BEGIN 4 */

/* USER CODE END 4 */
```

把中间改成：

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (rx_byte == '#')
        {
            send_enable = 0;  // 暂停发送
        }
        else if (rx_byte == '*')
        {
            send_enable = 1;  // 恢复发送
        }

        // 重新开启下一字节的中断接收
        HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
    }
}
```

注意：此代码依赖前一步已经在 `USER CODE BEGIN PV` 中定义了 `rx_byte` 和 `send_enable`，并在 `USER CODE BEGIN 2` 中调用了：

```
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
```

然后确认主循环发送部分是：

```
if (send_enable)
{
    HAL_UART_Transmit(&huart1, message, sizeof(message) - 1, HAL_MAX_DELAY);
}
```

保存并编译。

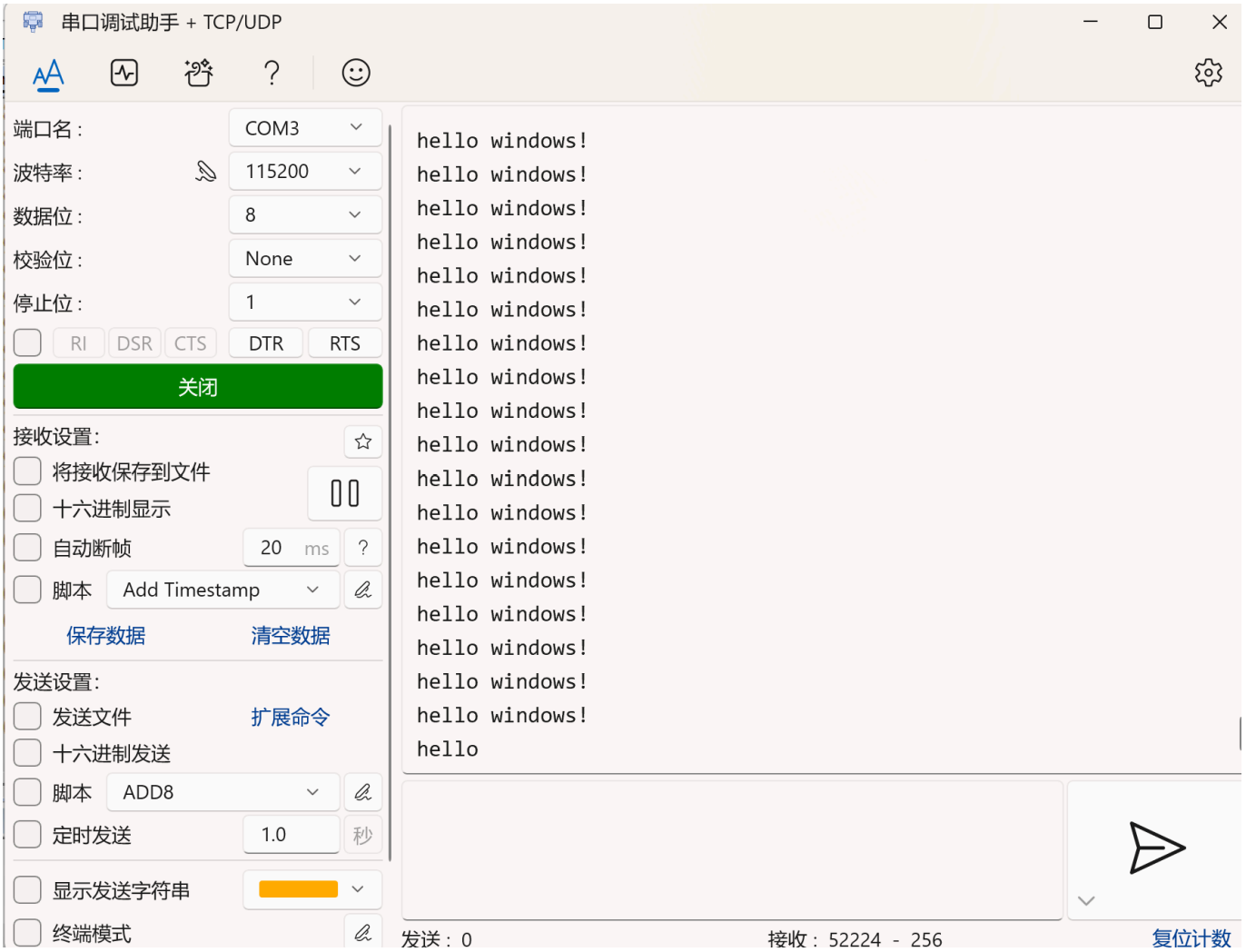
检查 `stm32f1xx_it.c` 里的 `USART1_IRQHandler()` 是否有 `HAL_UART_IRQHandler(&huart1);`

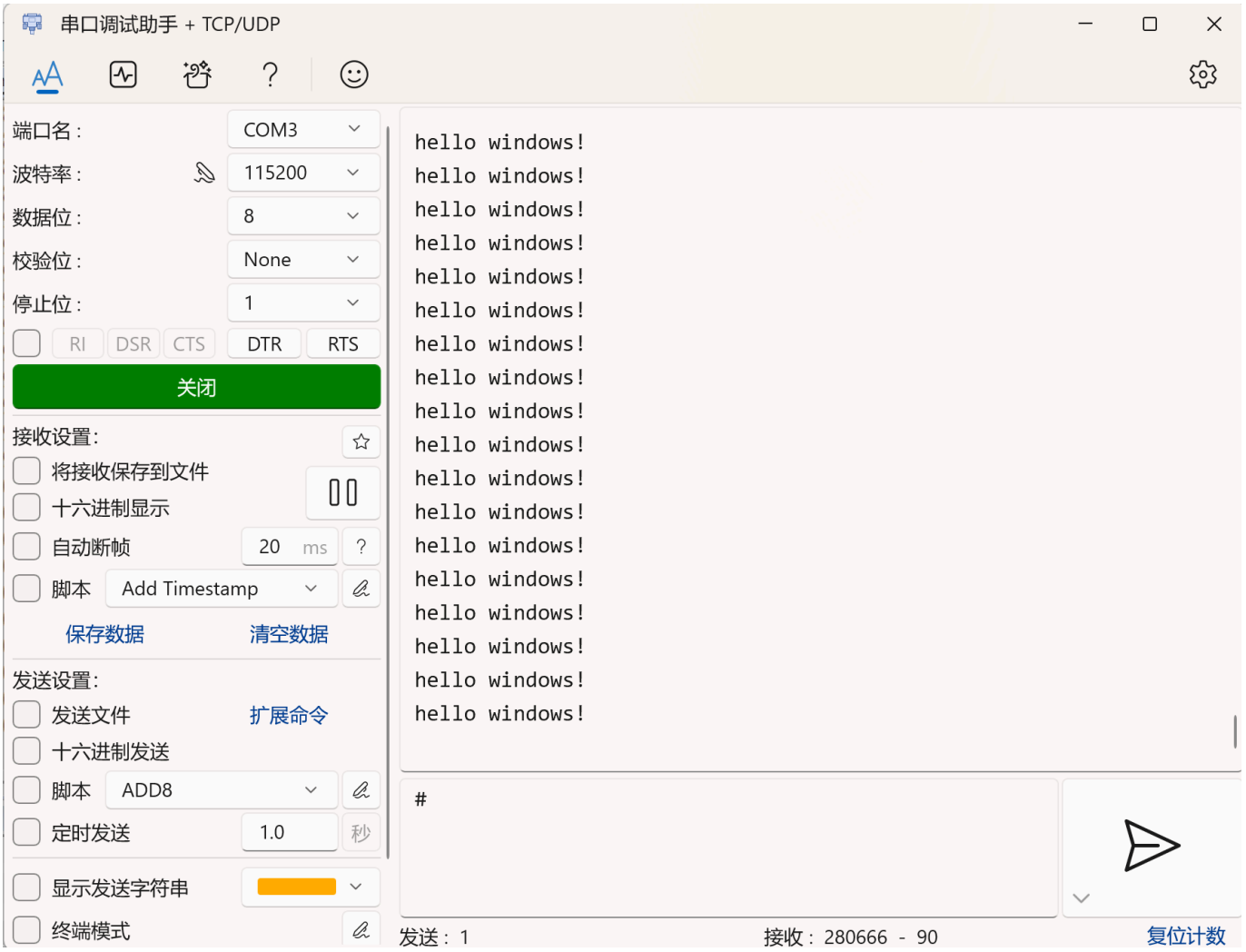
```
void USART1_IRQHandler(void)
{
  /* USER CODE BEGIN USART1_IRQn 0 */

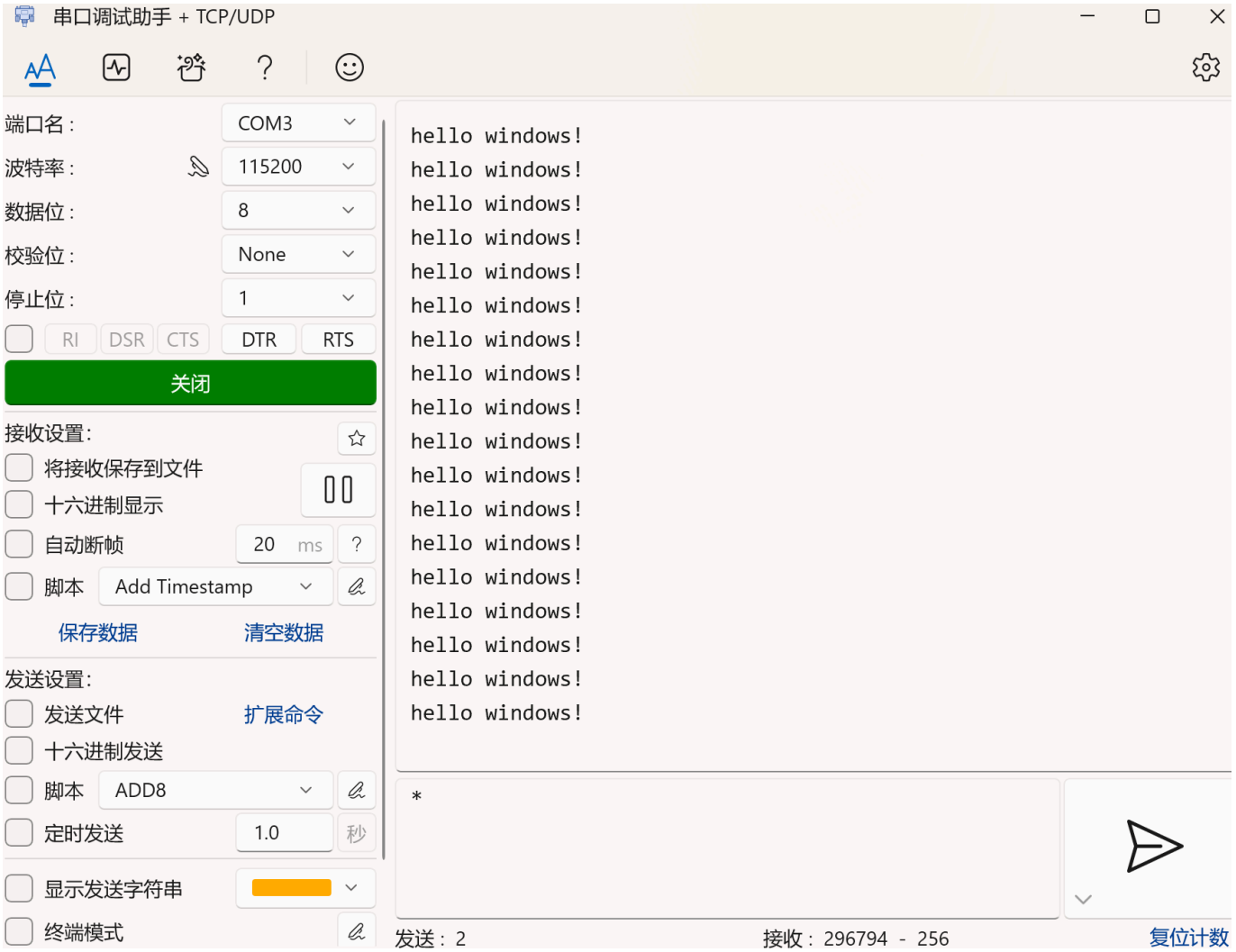
  /* USER CODE END USART1_IRQn 0 */
  HAL_UART_IRQHandler(&huart1);
  /* USER CODE BEGIN USART1_IRQn 1 */

  /* USER CODE END USART1_IRQn 1 */
}
```

观察结果： STM32 初始化 USART1 接收中断，通过 `HAL_UART_Receive_IT()` 接收上位机发送的单字节命令。在 `HAL_UART_RxCpltCallback()` 回调函数中，接收到字符 `#` 时将发送标志位置 0，暂停发送；接收到字符 `*` 时将发送标志位置 1，恢复发送。实验结果表明，发送 `#` 后串口助手不再接收到数据，发送 `*` 后恢复接收，功能正常。


串口调试助手 + TCP/UDP
端口名：
COM3
波特率：
115200
数据位：
8
校验位：
None
停止位：
1
RI
DSR
CTS
DTR
RTS
关闭
接收设置：
将接收保存到文件
十六进制显示
自动断帧
20
ms
脚本
Add Timestamp
保存数据
清空数据
发送设置：
发送文件
扩展命令
十六进制发送
脚本
ADD8
定时发送
1.0
秒
显示发送字符串
终端模式
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello
发送：0
接收：52224 - 256
复位计数

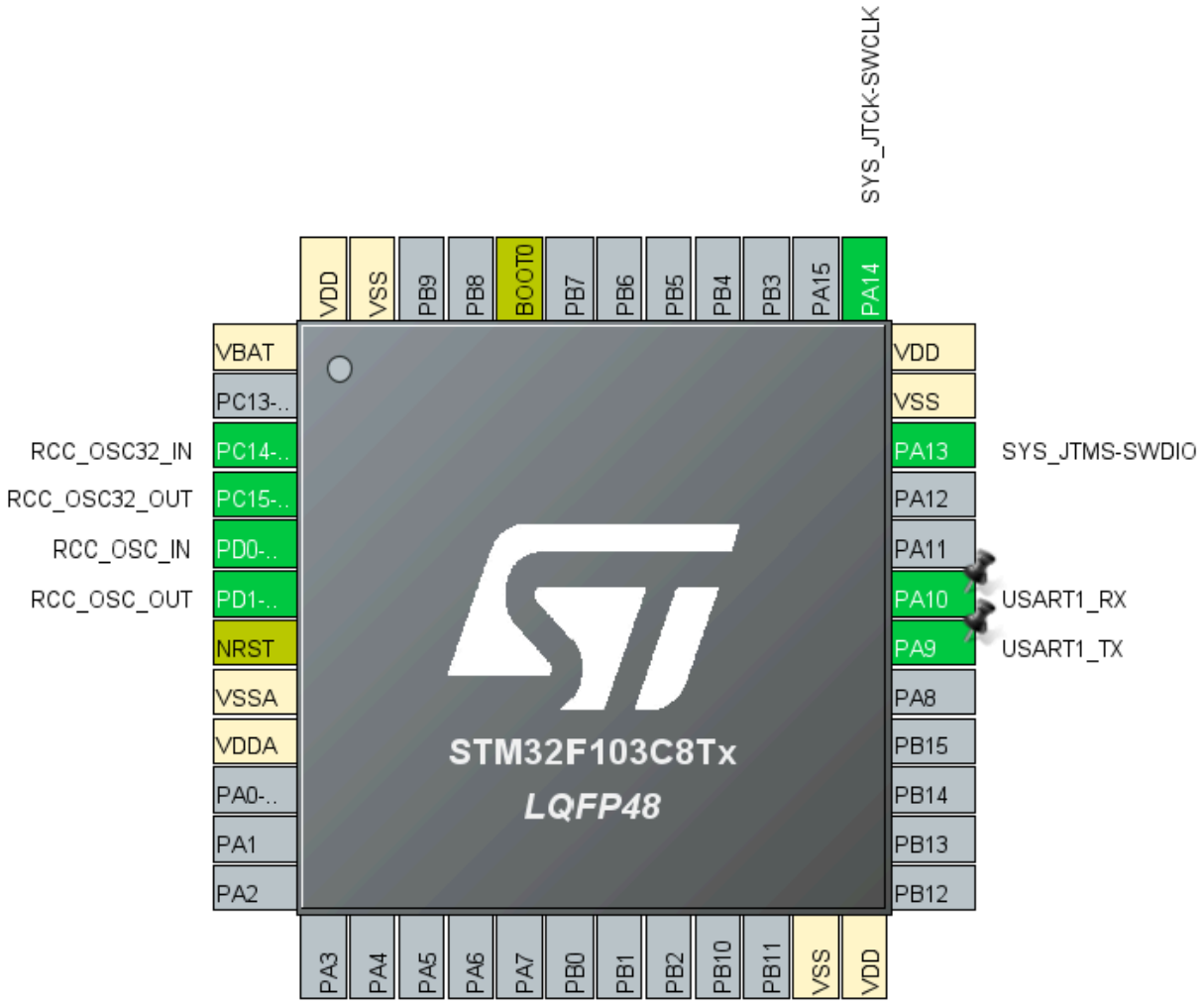

串口调试助手 + TCP/UDP
端口名：
COM3
波特率：
115200
数据位：
8
校验位：
None
停止位：
1
RI
DSR
CTS
DTR
RTS
关闭
接收设置：
将接收保存到文件
十六进制显示
自动断帧
20
ms
脚本
Add Timestamp
保存数据
清空数据
发送设置：
发送文件
扩展命令
十六进制发送
脚本
ADD8
定时发送
1.0
秒
显示发送字符串
终端模式
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
hello windows!
#
发送： 1
接收： 280666 - 90
复位计数

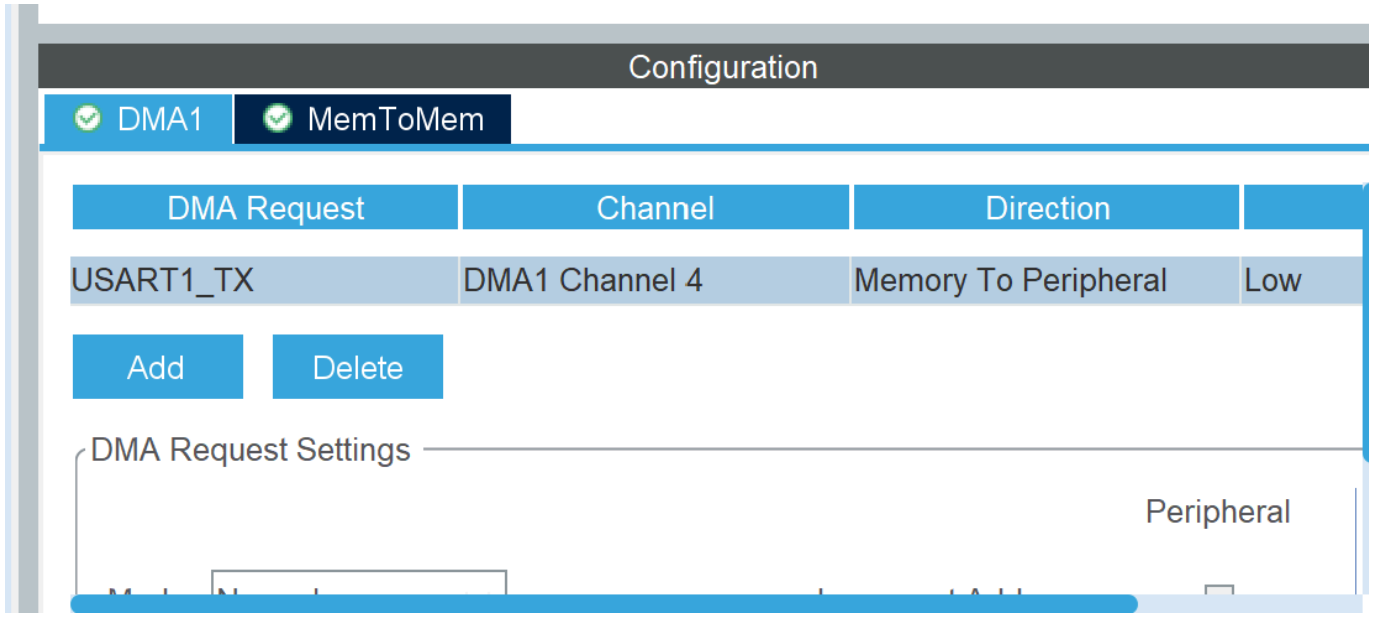


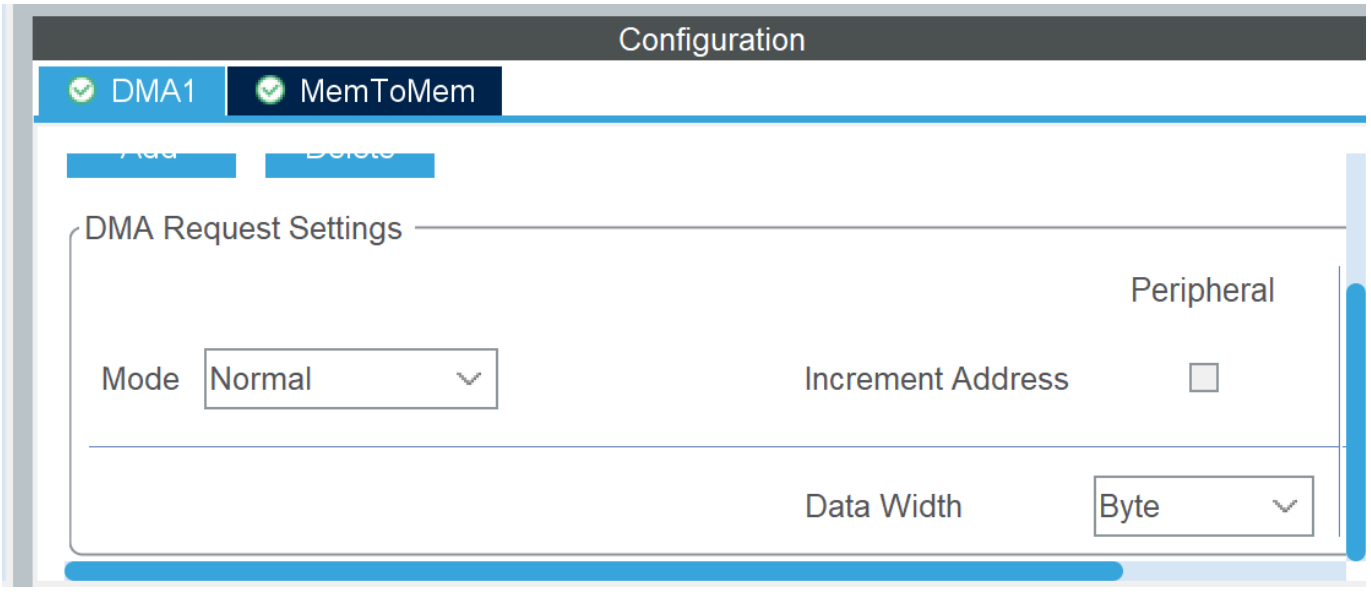
4.4 USART1_TX DMA 连续发送

在 CubeMX 里给 USART1 的 TX 配置 DMA：打开 USART1 的 **DMA Settings**，添加 **USART1_TX**，方向选 **Memory to Peripheral**，模式选 **Normal**，数据宽度保持 Byte。

- 打开 DMA1 Channel4 的中断；
- 在 NVIC 中勾选 DMA1 Channel4 global interrupt；
- USART1_TX 使用 DMA1 Channel4；
- DMA 模式选择 Normal；
- 发送完成后通过 `HAL_UART_TxCpltCallback()` 清除忙标志。


SYS_JTCK-SWCLK
VDD
VSS
PB9
PB8
BOOT0
PB7
PB6
PB5
PB4
PB3
PA15
PA14
VBAT
PC13-..
RCC_OSC32_IN
PC14-..
RCC_OSC32_OUT
PC15-..
RCC_OSC_IN
PD0-..
RCC_OSC_OUT
PD1-..
NRST
VSSA
VDDA
PA0-..
PA1
PA2
VDD
VSS
PA13
SYS_JTMS-SWDIO
PA12
PA11
PA10
USART1_RX
PA9
USART1_TX
PA8
PB15
PB14
PB13
PB12
STM32F103C8Tx
LQFP48
PA3
PA4
PA5
PA6
PA7
PB0
PB1
PB2
PB10
PB11
VSS
VDD


Configuration
DMA1
MemToMem
DMA Request
Channel
Direction
USART1_TX
DMA1 Channel 4
Memory To Peripheral
Low
Add
Delete
DMA Request Settings
Peripheral



在 `/* USER CODE BEGIN PV */` 中写：

```
uint8_t message[] = "hello windows!\r\n";
volatile uint8_t tx_busy = 0;
```

在 `/* USER CODE BEGIN 2 */` 中不用写发送代码，保持为空即可。

将 `while (1)` 中原来的：

```
HAL_UART_Transmit(&huart1, message, sizeof(message) - 1, HAL_MAX_DELAY);
```

替换为：

```
if (tx_busy == 0)
{
    tx_busy = 1;

    if (HAL_UART_Transmit_DMA(&huart1, message,
                              sizeof(message) - 1) != HAL_OK)
    {
        tx_busy = 0;
    }
}
```

然后在 `/* USER CODE BEGIN 4 */` 中加入 DMA 发送完成回调：

```c
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        tx_busy = 0;  // 本次DMA发送完成，允许下一次发送
    }
}
```

最后确认 `stm32f1xx_it.c` 中自动生成了下面这个函数，且不要删：

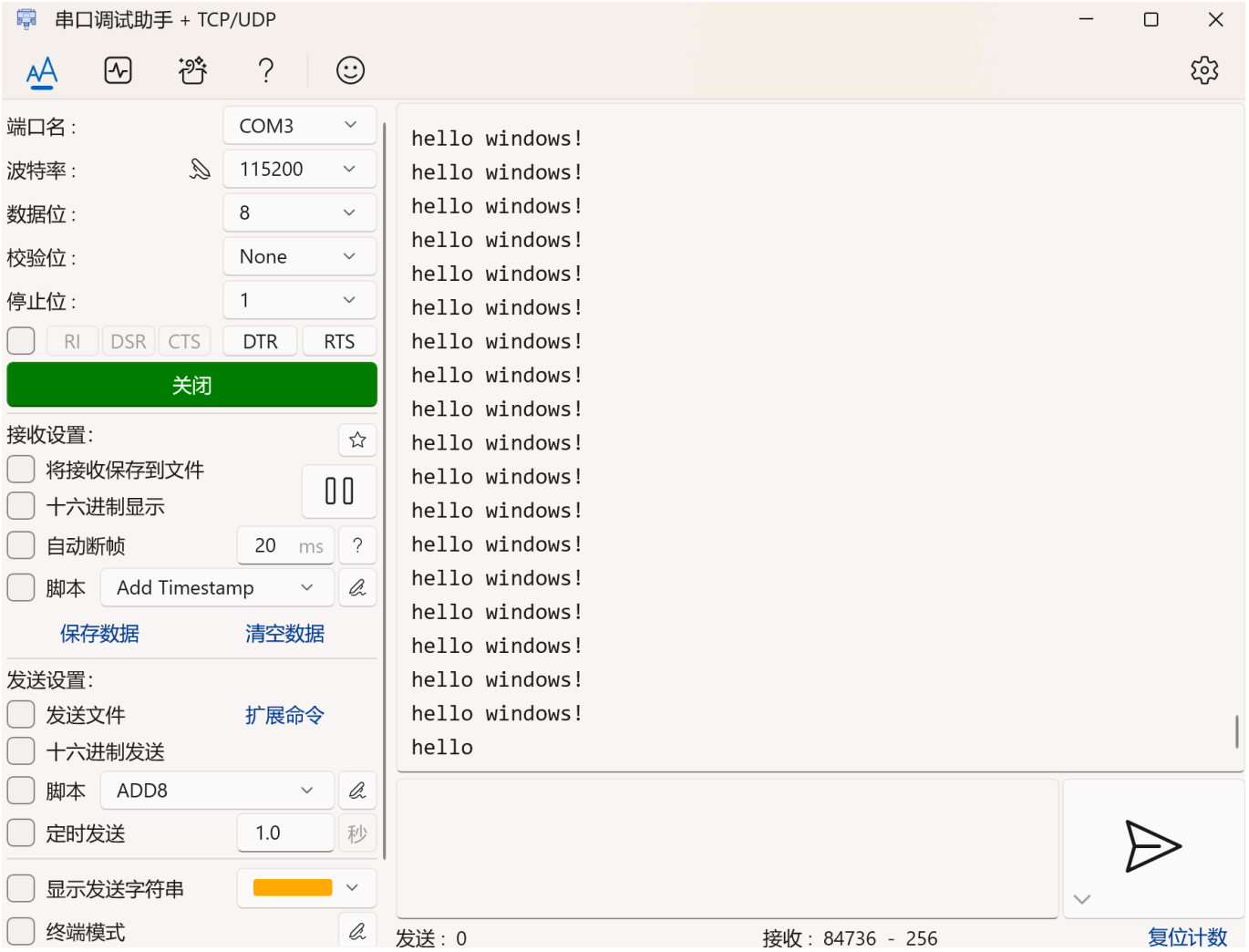
```c
void DMA1_Channel4_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&hdma_usart1_tx);
}
```

```c
void DMA1_Channel4_IRQHandler(void)
{
  /* USER CODE BEGIN DMA1_Channel4_IRQn 0 */

  /* USER CODE END DMA1_Channel4_IRQn 0 */
  HAL_DMA_IRQHandler(&hdma_usart1_tx);
  /* USER CODE BEGIN DMA1_Channel4_IRQn 1 */

  /* USER CODE END DMA1_Channel4_IRQn 1 */
}
```

编译、下载后，串口助手仍设为 `115200 / 8 / None / 1`。



4.5 双电脑串口文件传输

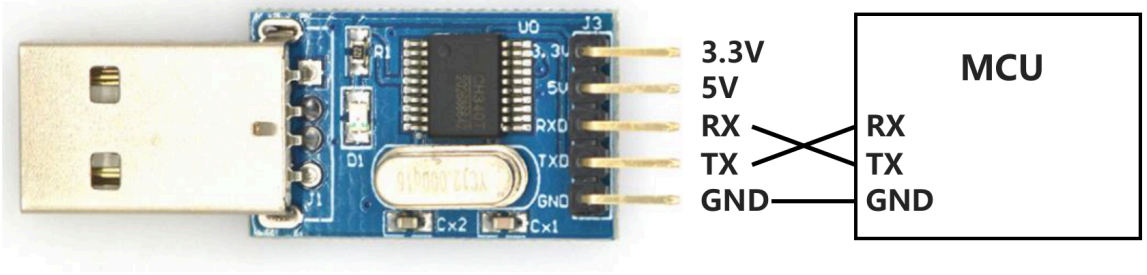
使用两块CH340 USB-TTL模块实现两台Windows电脑之间TTL串口直连，利用串口助手完成二进制文件传输，对比**理论传输时间**与**实际传输时间**，理解串口实际吞吐受软件、缓冲区影响的特点。

4.5.1 硬件接线

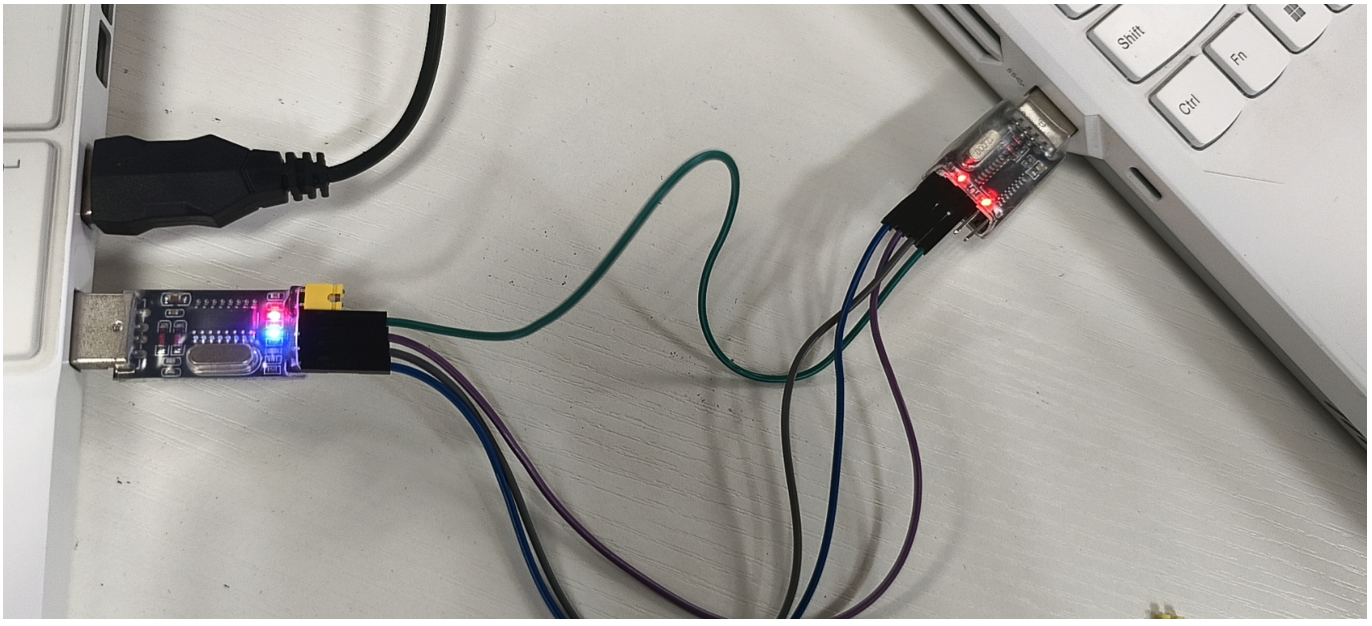
两块USB-TTL模块交叉互连：模块A的TXD接模块B的RXD；模块A的RXD接模块B的TXD；两块模块的**GND必须相连共地**；VCC不需要互相连接。两个模块分别插入两台电脑USB接口。

串口转换模块

USB转TTL串口模块



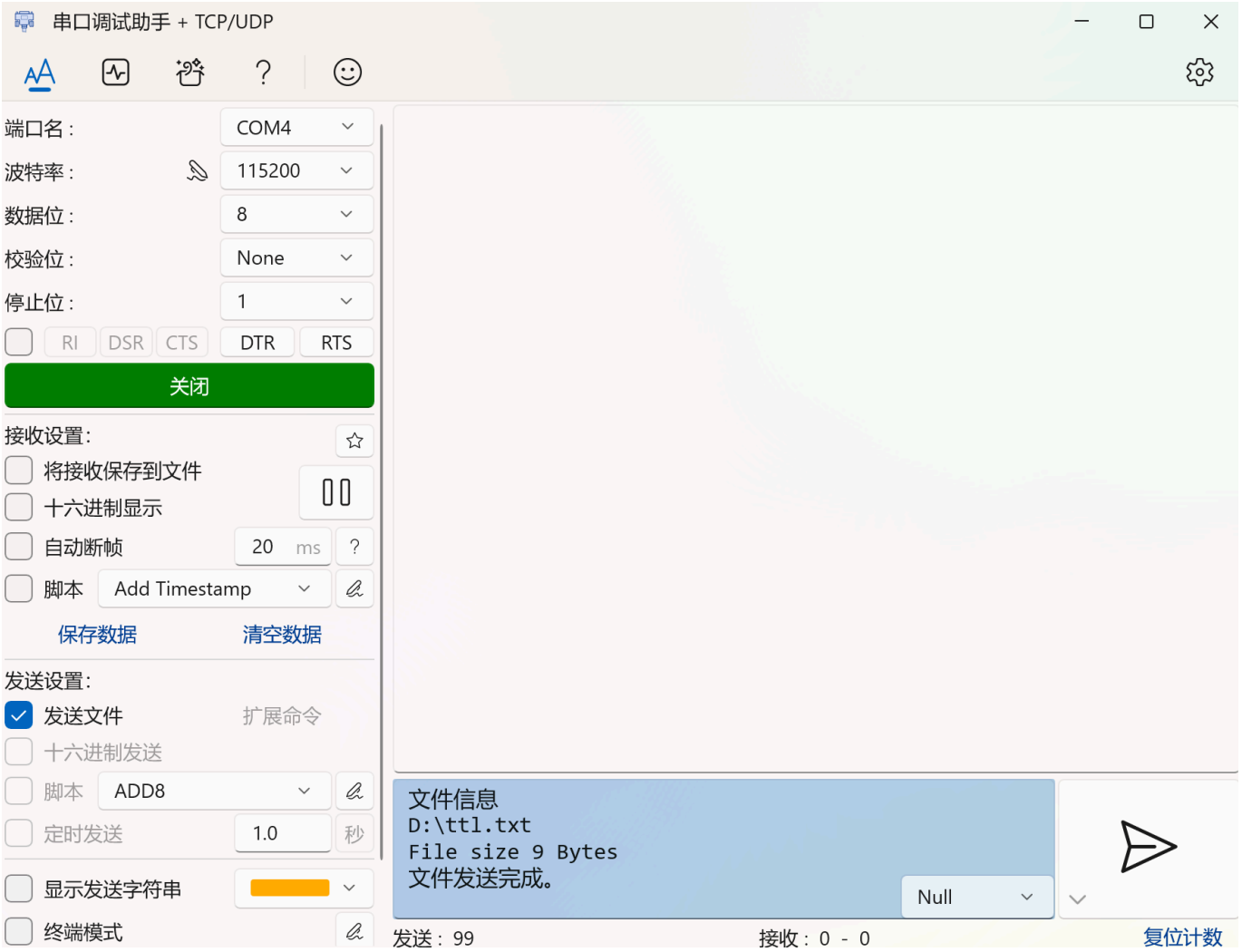
- MCU的TX引脚和串口转换模块的RX引脚连接
- MCU的RX引脚和串口转换模块的TX引脚连接
- MCU的地和串口转换模块的地连接，以确保电平的一致



4.5.2 软件参数配置

两台电脑均打开串口助手，选择各自设备管理器识别到的COM口号，双方配置参数必须完全一致：

波特率：115200
数据位：8
停止位：1
校验位：None
流控：None
关闭十六进制发送
关闭自动追加回车换行
接收端勾选【保存接收数据到文件】



4.5.3 实验操作步骤

1. 两台电脑分别打开对应COM串口，确认串口打开成功；
2. **接收电脑**提前开启“保存接收数据到文件”，设置输出保存路径；
3. **发送电脑**点击【打开文件】，选取待传输测试文件，记录文件字节大小S；
4. 点击【发送文件】同时使用秒表计时，记录传输完成的实际总时间T_{actual}；
5. 传输结束，在接收电脑找到生成的接收文件；对比源文件与接收文件大小，校验文件完整性；

6. 根据公式 $T_{ideal} = \dfrac{10S}{B}$ 计算115200bps下理论线路传输时间；将理论时间与实测时间填入下表。

4.5.4 实验数据记录

参数项	数值
波特率 B	115200 bps
原始文件大小 S（字节）	1879034
理论传输时间 $T_{ideal} = \dfrac{10S}{B}$	163.19 s
实测传输时间 T_{actual}	172.5 s
接收文件字节大小	1879034
文件是否完整一致	是

4.5.5 现象与分析

1. 硬件连接错误（未交叉TX-RX，或者**没有接GND**）会出现完全收不到数据、乱码、传输不稳定现象。TTL单端通信必须共地，否则电平参考基准不一致，无法正确识别高低电平。
2. 实测传输时间T_{actual}一般大于理论T_{ideal}。理论公式仅计算比特在线路上传输耗时；实际串口助手存在分包、软件延时、USB驱动缓冲、操作系统调度开销，会拉长总传输时间。
3. 如果收发双方波特率、数据位、停止位、校验位设置不一致，收到的全部是乱码，文件无法复原。
4. 对比源文件和接收文件属性：字节数完全一致代表传输完整；字节数不一致说明发生丢字节。

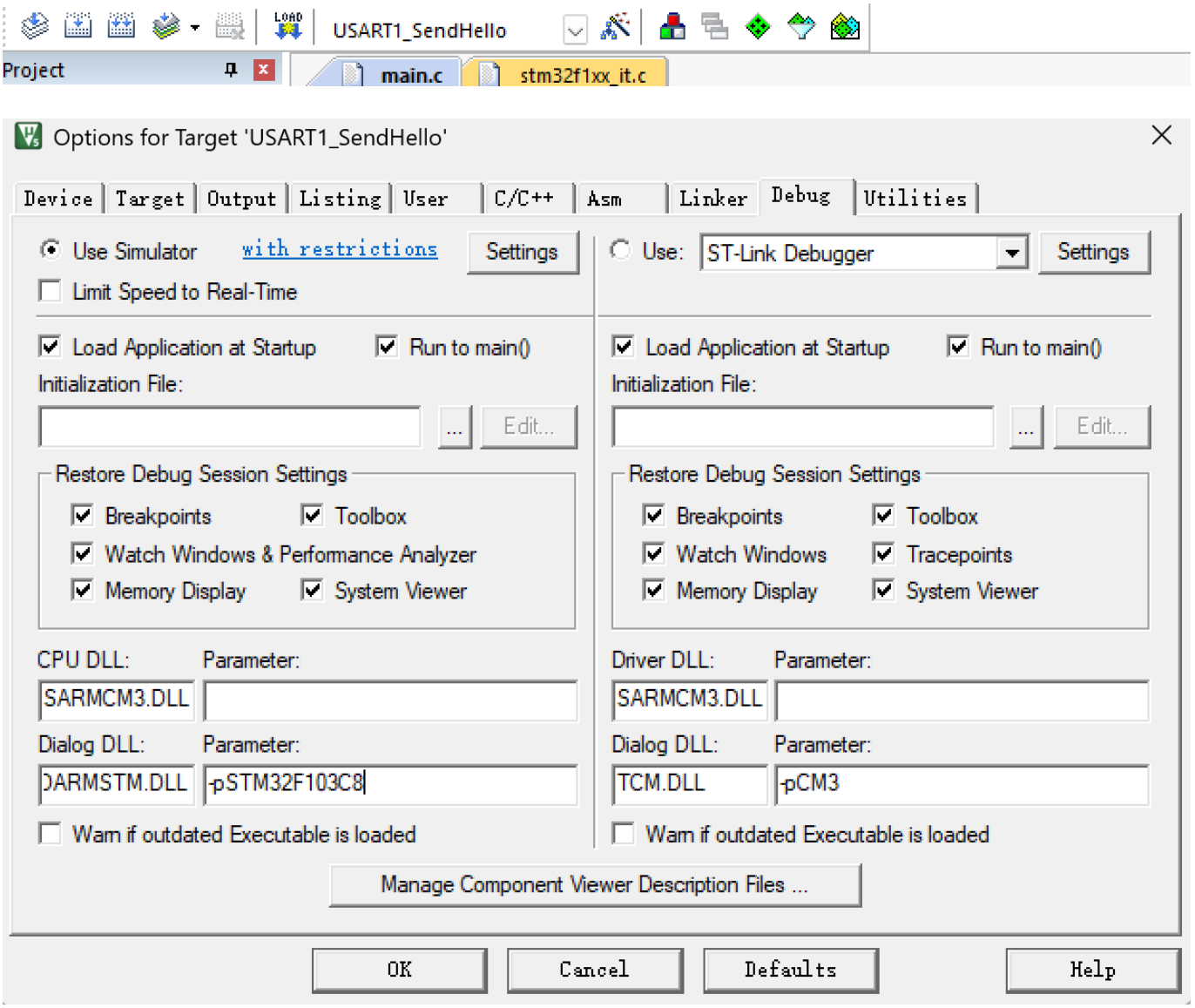
4.5.6 注意事项

- 不能使用RS-232电平模块直接接到TTL的TX/RX，会损坏CH340；本实验全程为TTL电平；
- 传输二进制文件，务必关闭自动加回车换行、关闭十六进制模式，否则原始数据被篡改，接收文件损坏；
- 文件校验建议比对文件字节大小，有条件可做MD5校验，不能只看串口助手窗口显示。

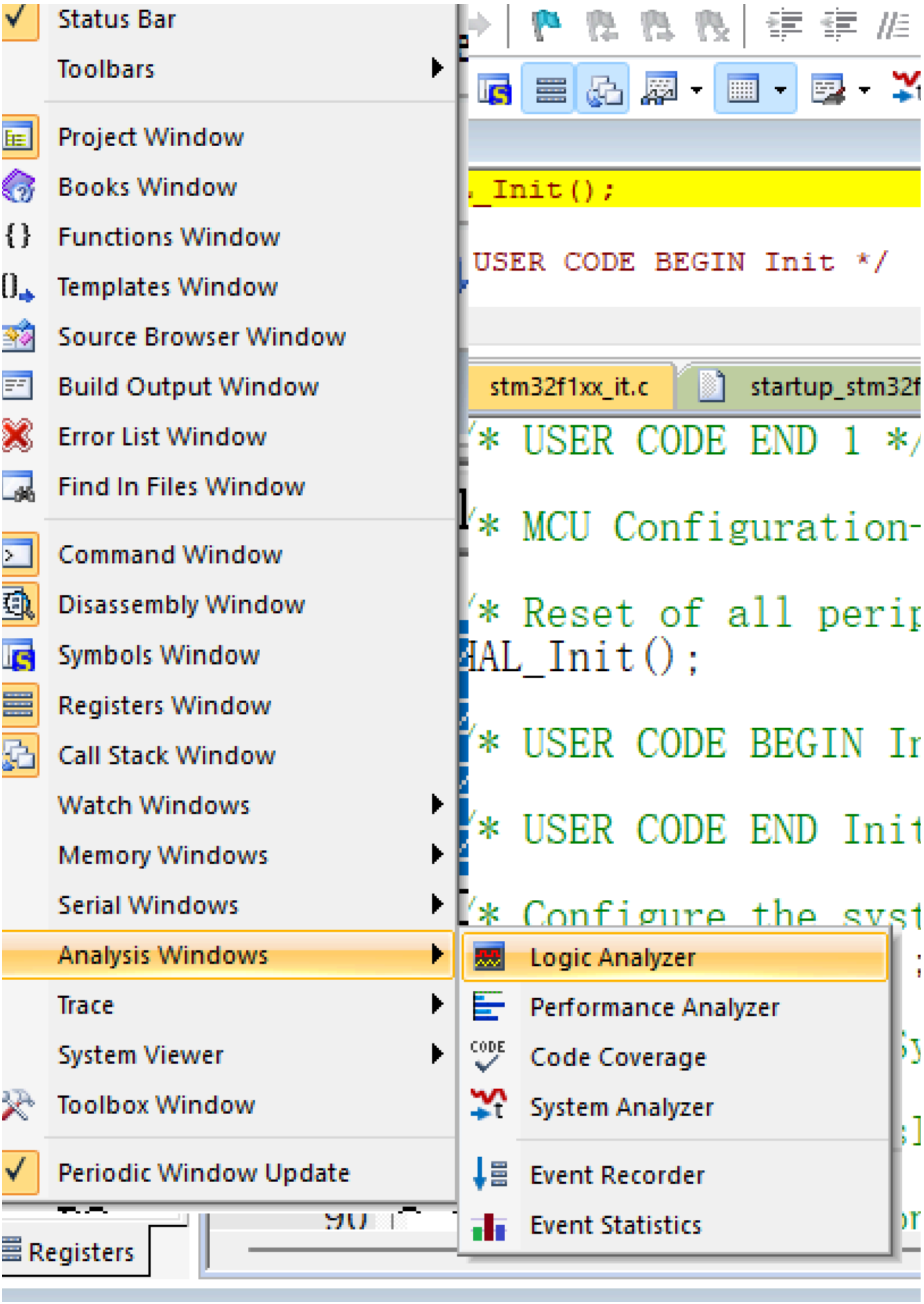
4.6 Keil Logic Analyzer 波形尝试

- 点击 `Options for Target`（魔术棒）。
- 打开 `Debug` 选项卡。
- 选择 `Use Simulator`，不要选 ST-Link。
- 点击确定。

- 点击 `Debug → Start/Stop Debug Session` 进入仿真。

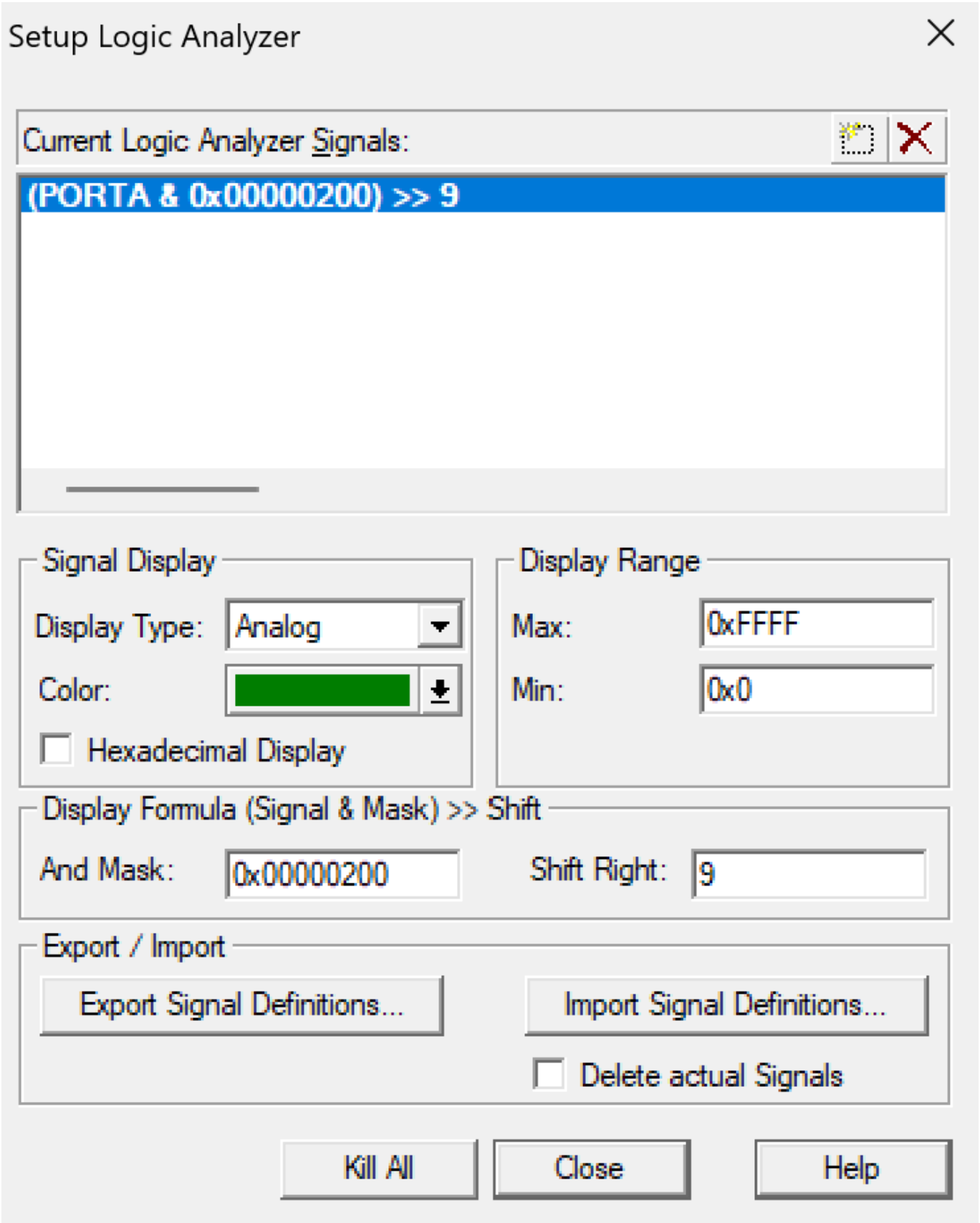


点击菜单 `View → Analysis Windows → Logic Analyzer`，打开逻辑分析仪窗口。

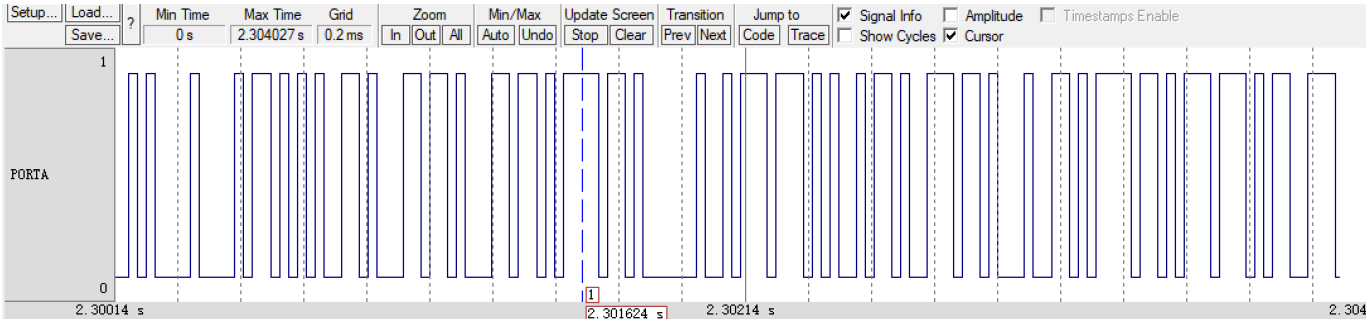

Status Bar
Toolbars
Project Window
Books Window
Functions Window
Templates Window
Source Browser Window
Build Output Window
Error List Window
Find In Files Window
Command Window
Disassembly Window
Symbols Window
Registers Window
Call Stack Window
Watch Windows
Memory Windows
Serial Windows
Analysis Windows
Trace
System Viewer
Toolbox Window
Periodic Window Update
Logic Analyzer
Performance Analyzer
Code Coverage
System Analyzer
Event Recorder
Event Statistics
Registers
_Init();
USER CODE BEGIN Init */
stm32f1xx_it.c
startup_stm32f
/* USER CODE END 1 */
/* MCU Configuration-
/* Reset of all perip
HAL_Init();
/* USER CODE BEGIN In
/* USER CODE END Init
/* Configure the syst

在逻辑分析仪窗口中，点击 `Setup...` 或右键空白处选择添加信号。添加表达式：

```
(PORTA & 0x00000200) >> 9
```



仿真运行后，Logic Analyzer 采集 PA9 引脚电平，空闲状态为高电平；发送数据时，会出现 UART 帧对应的电平跳变，依次产生起始位、数据位和停止位波形。启动仿真运行，Logic Analyzer 捕获 PA9 引脚波形，波形可见 UART 帧对应的电平跳变，包含起始位、8bit 数据位与停止位。根据 APB2 时钟和 USART1 的 BRR 配置，理论波特率为 115200 bps，理论位时间为 8.68 μs。Logic Analyzer 波形可用于观察 8N1 的高低电平变化，证明 USART1 波特率配置正确，串口时序符合 8N1 异步通信规范。



APB2 时钟 = 72 MHz USART1 波特率设置 = 115200 bps USARTDIV = 72,000,000 ÷ (16 × 115200) = 39.0625 BRR = 0x0271 实际波特率 = 72,000,000 ÷ (16 × 39.0625) = 115200 bps 单比特时间 = 1 ÷ 115200 ≈ 8.68 μs

8N1 每个字符包含 1 位起始位、8 位数据位、1 位停止位，共 10 位，所以一个字符约：

10 × 8.68 μs = 86.8 μs

所以配置推算的波特率为 **115200 bps**，理论位宽为 **8.68 μs**。

5. 误差与问题分析

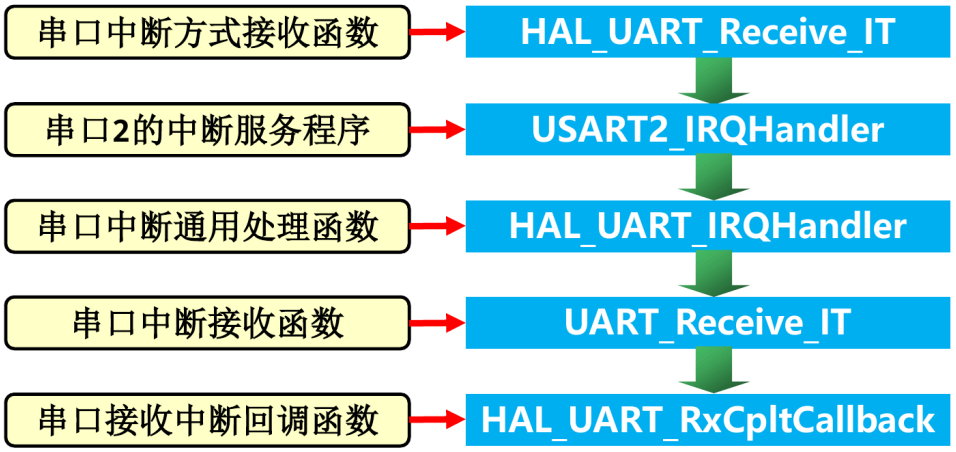
1. **波特率不匹配：** 异步串口双方依靠共同波特率采样。任一端设置不同，字符可能出现乱码或帧错误。高波特率测试时应确认 STM32 已重新生成、编译并下载相同设置，串口助手也设置到同一波特率。
2. **文件传输时间偏长：** 实际软件可能加入分包、校验、应答、等待间隔和重传；USB 驱动、串口缓冲区和接收软件也会影响吞吐率。
3. **高速接收显示：** 串口助手窗口滚动刷新不等于可靠性统计。需要保存原始接收数据并做长度/内容/校验比较。

   4.**共地要求：** USB-TTL 的单端信号以地为参考。省略 GND 后可能因两端地电位不确定而造成不稳定接收。

6. 结论

串口中断过程

串口中断处理过程



本实验完成 STM32F103C8Tx 芯片 USART1 的 CubeMX 与 HAL 库配置，借助 USB-TTL 模块与 Windows 串口助手，在 115200 波特率、8N1 帧格式下实现串口通信。编写的轮询发送程序可以持续向上位机发送字符串`hello windows!`；串口接收中断程序实现简单控制功能：接收到字符`#`暂停发送，接收到`*`恢复发送。同时完成 USART1_TX 的 DMA 配置，利用 DMA1 Channel4 实现数据连续发送，上位机串口助手能够正常接收传输的数据。通过实验我理解了 DMA 的核心优势：减少 CPU 在数据搬运工作中的占用，不过串口通信的波特率，依旧需要 USART 外设和上位机串口助手双方保持一致才能正常通信。

本次 Keil Logic Analyzer 软件仿真成功捕获 PA9 引脚的 UART 时序波形。基于 72 MHz 的 APB2 时钟与 BRR 寄存器配置值`0x0271`，可计算得到配置的理论波特率为 115200 bps，单个比特的理论时间为 8.68 μs。我们可以在 Logic Analyzer 中利用游标测量波形的实际位宽，将实测值与理论值相互对照。